

End-to-End AQI Predictor for Faisalabad

1. System Overview & Core Objectives

The objective of this project is to build an automated, serverless End-to-End Machine Learning Platform to forecast the Air Quality Index (AQI) for the next 3 days. The platform integrates automated data ingestion, feature engineering, model retraining, continuous integration/deployment (CI/CD), and a decoupled web application.

2. Technology Stack Mapping

- **Developer & CI/CD Pipeline:** GitHub Repository, GitHub Actions (Data orchestration & deployment)
- **Data Ingestion:** OpenWeather Air Pollution & Weather APIs
- **Feature Store & Model Registry:** Hopsworks Feature Store & Hopsworks Model Registry
- **Machine Learning & Analytics:** Scikit-learn, TensorFlow/PyTorch, SHAP
- **Backend Serving:** FastAPI (Docker containerized)
- **Frontend Web Application:** React (HTML, CSS, JS)

3. Detailed Functional & Component Requirements

Module 1: Developer Workspace & CI/CD Pipeline

- **Repository Management:** GitHub serves as the central source code repository.
- **Continuous Integration & Automation:** GitHub Actions workflows trigger on code pushes and on cron schedules to execute automated testing, feature processing, model training, and container builds.

Module 2: Data Orchestration & Feature Pipeline (Hourly Execution)

- **Data Extraction:** An automated hourly cron workflow in GitHub Actions fetches raw weather parameters (temperature, humidity, wind speed) and atmospheric pollutant levels (PM_{2.5}, PM₁₀, NO₂, SO₂, CO, O₃) from OpenWeather APIs.
- **Preprocessing & Feature Engineering:** Clean raw records, handle missing entries, extract temporal features (hour, day, month, day of week), and compute derived features (rolling averages, AQI change rates).
- **Feature Persistence:** Ingest processed feature vectors directly into the Hopsworks Feature Store feature group.
- **Historical Backfill:** Execute a backfill script during initialization to populate the Hopsworks Feature Store with historical data for model training.

Module 3: Model Training & Evaluation Pipeline (Daily Execution)

- **Feature Retrieval:** Fetch historical feature sets and target values from Hopsworks Feature Store.
- **Model Training:** Train multiple model types, including Scikit-learn regression algorithms (Random Forest, Ridge) and Deep Learning architectures (TensorFlow/PyTorch).
- **Model Evaluation:** Measure performance using RMSE, MAE, and R^2 metrics. Compute SHAP values for model explainability.
- **Model Registration:** Compare the newly trained model against baseline metrics. Automatically register the best-performing model into the Hopsworks Model Registry.

Module 4: Application Serving Layer

- **FastAPI Backend:**
 - Pulls the latest registered model artifact from the Hopsworks Model Registry at runtime.
 - Queries recent feature vectors from the Hopsworks Feature Store.
 - Generates 3-day AQI prediction payloads exposed via RESTful JSON endpoints.
 - Implements threshold checks to return hazard alerts if predicted AQI values breach safe levels.
- **React Frontend Dashboard:**
 - Consumes JSON responses from the FastAPI REST backend.
 - Renders 3-day AQI forecasting charts, hourly pollutant trends, and SHAP explainability insights.

4. Data Flow Logic

1. **Hourly:** GitHub Actions triggers the ETL workflow → OpenWeather API → Extraction & Feature Engineering → Ingestion to Hopsworks Feature Store.
2. **Daily:** GitHub Actions triggers the Training workflow → Ingest historical feature data from Hopsworks → Train & evaluate models → Register top model to Hopsworks Model Registry.
3. **On-Demand (User Request):** React Frontend sends a request to FastAPI Backend → FastAPI fetches features and model from Hopsworks → Runs inference → Returns JSON predictions & SHAP explanations to the React Dashboard.